

Ruprecht-Karls-Universität Heidelberg
Institut für Informatik

Bachelor Thesis
Parallel ASP Planning

Name: Levi Maximilian Klein
Student ID: 4283957
Supervisor: Dr. Gnad
Submission date: 17.02.2026

Ich versichere, dass ich diese Bachelor-Arbeit selbstständig verfasst und nur die angegebenen Quellen und Hilfsmittel verwendet habe.

Heidelberg, dd.mm.yyyy

Zusammenfassung

Dies ist eine Zusammenfassung der Arbeit.

Abstract

This is the abstract.

Contents

List of Figures	1
List of Tables	2
1 Introduction	3
1.1 Motivation	3
1.2 Goal of the Thesis	3
1.3 Structure of the Thesis	3
2 Theory	4
2.1 Definition	4
2.2 ASP	6
2.2.1 encoding for sequential plans	8
2.2.2 encoding for $\forall$ -step plans	8
Bibliography	9

List of Figures

List of Tables

1 Introduction

This chapter contains an overview of the topic as well as the goals and contributions of your work.

1.1 Motivation

1.2 Goal of the Thesis

1.3 Structure of the Thesis

Here you describe the structure of the thesis. For example:

In Kapitel 2 werden grundlegende Methoden für diese Arbeit vorgestellt.

2 Theory

2.1 Definition

Definition 1 *Multi-valued planning task*. A STRIPS-like multivalued planning task according to (Helmert 2006) is given by a 4 tuple $\langle \mathcal{F}, s_0, s_*, \mathcal{O} \rangle$, where:

- $\mathcal{F}$ is a finite set of state variables, also called **fluents**. Each $x \in \mathcal{F}$ has a finite domain x^d and in every state, x takes exactly one value from its domain.
- s_0 , the **initial state**, is a total function, s.t. $s_0(x) \in x^d$ for each $x \in \mathcal{F}$
- s_* , the **goal conditions**, is a partial function, s.t. $s_*(x) \in x^d$, for each $x \in \text{dom}(s_*)$
- $\mathcal{O}$ is a finite set of operators, also called **actions**. We define an action as $a = \langle a^c, a^e \rangle$, where a^c is the precondition of a and a^e is the postcondition of a . Both a^c and a^e are partial variable assignments over $\mathcal{F}$.

Example 1. Let us illustrate the definitions with a small example. The plan is to leave the house by opening the door, going outside, and finally closing the door.

- $\mathcal{F} = \{\text{door}, \text{location}\}$ with $\text{door}^d = \{\text{open}, \text{closed}\}$, $\text{location}^d = \{\text{inside}, \text{outside}\}$
- Initial state: $s_0 = \{\text{door}=\text{closed}, \text{location}=\text{inside}\}$
- Goal: $s_* = \{\text{door}=\text{closed}, \text{location}=\text{outside}\}$
- Actions: $\mathcal{O} = \{\text{openDoor}, \text{closeDoor}, \text{goOut}\}$, where

$$\begin{aligned} \text{openDoor} &= \langle \{\text{door}=\text{closed}\}, \{\text{door}=\text{open}\} \rangle \\ \text{closeDoor} &= \langle \{\text{door}=\text{open}\}, \{\text{door}=\text{closed}\} \rangle \\ \text{goOut} &= \langle \{\text{location}=\text{inside}, \text{door}=\text{open}\}, \{\text{location}=\text{outside}\} \rangle \end{aligned}$$

Definition 2 *Successor state*. Let s be state and $a \in \mathcal{O}$ an action.

The *successor state* $o(a, s)$, obtained by applying action a in state s , is defined if the precondition of a holds, i.e., $a^c(x) = s(x)$ for all $x \in \text{dom}(a^c)$. Then

$$s'(x) = o(a, s) = \begin{cases} a^e(x) & \text{if } x \in \text{dom}(a^e) \\ s(x) & \text{otherwise.} \end{cases}$$

Remark. Let $A_n = \langle a_1, \dots, a_n \rangle$ be a sequence of actions with length n . We define the application

of A_n to a state s recursively by

$$o(A_n, s) = o(a_n, o(A_{n-1}, s))$$

if $o(A_i, s)$ is defined for all $1 \leq i \leq n$.

Definition 3 Sequential plan. A *sequential plan* is a sequence of actions $A_n = \langle a_1, \dots, a_n \rangle$, s.t. $s' = o(A_n, s_0)$ is defined and $s'(x) = s_*(x)$ for all $x \in \text{dom}(s_*)$.

Example 2. Consider the planning task from the previous example. A sequential plan would be $A = \langle \text{openDoor}, \text{goOut}, \text{closeDoor} \rangle$. Since

$$\begin{aligned} o(A, s_0) &= o(\langle \text{openDoor}, \text{goOut}, \text{closeDoor} \rangle, \{\text{door}=\text{closed}, \text{location}=\text{inside}\}) \\ &= o(\langle \text{goOut}, \text{closeDoor} \rangle, \{\text{door}=\text{open}, \text{location}=\text{inside}\}) \\ &= o(\text{closeDoor}, \{\text{door}=\text{open}, \text{location}=\text{outside}\}) \\ &= \{\text{door}=\text{closed}, \text{location}=\text{outside}\} = s_* \end{aligned}$$

To get to parallel actions and plans, we need the definition of confluent actions.

Definition 4 confluent. Let $A = \{a_1, \dots, a_n\} \subseteq \mathcal{O}$ be a set of actions.

A is *confluent* $:\Leftrightarrow a_i^e(x) = a_j^e(x)$ for all $1 \leq i < j \leq n$ and each $x \in \text{dom}(a_i^e) \cap \text{dom}(a_j^e)$

Remark. A is confluent $\Leftrightarrow B$ is confluent for all $B \subseteq A$

Example 3. In our example $\{\text{openDoor}, \text{goOut}\}$ and $\{\text{closeDoor}, \text{goOut}\}$ are confluent sets.

We extend our example with the fluents *light* and *key* with $\text{light}^d = \text{key}^d = \{0, 1\}$ and two actions *lightsOut* and *getKey*:

$$\begin{aligned} \text{lightsOut} &= \langle \{\text{location}=\text{inside}, \text{light}=1\}, \{\text{light}=0\} \rangle \\ \text{getKey} &= \langle \{\text{location}=\text{inside}, \text{key}=0\}, \{\text{key}=1\} \rangle \end{aligned}$$

Additionally, we modify the precondition: $\text{openDoor}^c = \{\text{door}=\text{closed}, \text{key}=1\}$. Our new goal is $s_* = \{\text{location}=\text{outside}, \text{door}=\text{closed}, \text{light}=0\}$.

Now for example $\{\text{getKey}, \text{openDoor}, \text{goOut}, \text{lightsOut}\}$ and $\{\text{getKey}, \text{closeDoor}, \text{goOut}, \text{lightsOut}\}$ are also confluent sets.

Definition 5 Parallel representations. Let s be a state and $A = \{a_1, \dots, a_n\}$ be a confluent set of actions. Then A is

- (i) $\forall$ -step serializable in $s : \Leftrightarrow o(\langle a_{\pi(1)}, \dots, a_{\pi(n)} \rangle, s)$ is defined for all permutations π
- (ii) $\exists$ -step serializable in $s : \Leftrightarrow o(\langle a_{\pi(1)}, \dots, a_{\pi(n)} \rangle, s)$ is defined for some permutation $\pi[t]$ and $a^c(x) = s(x)$ for each $a \in A$ and $x \in \text{dom}(a^c)$
- (iii) relaxed $\exists$ -step serializable in $s : \Leftrightarrow o(\langle a_{\pi(1)}, \dots, a_{\pi(n)} \rangle, s)$ is defined for some $[t]$ permutation π

Remark. A is $\forall$ -step serializable $\Rightarrow A$ is $\exists$ -step serializable $\Rightarrow A$ is relaxed $\exists$ -step serializable.

Definition 6 *Parallel Plans.* Let $A = \langle A_1, \dots, A_n \rangle$ be a sequence of confluent sets of actions. A is a $\forall$ -step, $\exists$ -step or relaxed $\exists$ -step plan iff

- $s_n(x) = s_*(x)$ for each $x \in \text{dom}(s_*)$
- A_i is $\forall$ -step, $\exists$ -step or relaxed $\exists$ -step serializable in s_{i-1} for $1 \leq i \leq n$, where

$$s_i(x) = \begin{cases} a^e(x) & \text{if } x \in \text{dom}(a^e) \\ s_{i-1}(x) & \text{else} \end{cases} \quad \text{for each } a \in A_i$$

Example 4. In our modified example we have:

- $\forall$ -step plan: $\langle \{getKey, lightsOut\}, \{openDoor\}, \{goOut\}, \{closeDoor\} \rangle$
- $\exists$ -step plan: $\langle \{getKey, lightsOut\}, \{openDoor\}, \{goOut, closeDoor\} \rangle$
- relaxed $\exists$ -step plan: $\langle \{getKey, lightsOut, openDoor, goOut\}, \{closeDoor\} \rangle$

2.2 ASP

Idee: Alles mit den mathematische Formeln von oben beschreiben. In ASP we represent planning tasks by facts. A solution of an ASP encoding is called stable model.

```

1  fluent(door; loc; light; key).
2  value(door, 0).  value(door, 1).
3  value(loc, in).  value(loc, out).
4  value(light, 0). value(light, 1).
5  value(key, 0).   value(key, 1).
6
7  init(door, 0).  init(loc, in).  init(light, 1).  init(key, 0).
8  goal(door, 0).  goal(loc, out).  goal(light, 0).
9
10 action(openDoor; closeDoor; goOut; lightOut; getKey).
11 prec(openDoor, door, 0).  prec(openDoor, key, 1).  post(openDoor, door, 1).
12 prec(closeDoor, door, 1).  post(closeDoor, door, 0).
13 prec(goOut, loc, in).  prec(goOut, door, 1).  post(goOut, loc, out).
14 prec(lightOut, loc, in).  prec(lightOut, light, 1).  post(lightOut, light, 0).
15 prec(getKey, loc, in).  prec(getKey, key, 0).  post(getKey, key, 1).

```

Listing 1: ASP encoding of the example planning task

Combining these facts with the right encodings, stable models corresponds to the *sequential*, $\forall$ -step, $\exists$ -step or relaxed $\exists$ -step plan.

```

1  #include <incmode>.
2  holds(X, V, 0) :- init(X, V).
3  #program check(t).
4  :- query(t), goal(X, V), not holds(X, V, t).
5  #program step(t).
6  {holds(X, V, t) : value(X, V)} = 1 :- fluent(X).
7  {occurs(A, t)} :- action(A).
8  :- occurs(A, t), post(A, X, V), not holds(X, V, t).
9  :- holds(X, V, t), not holds(X, V, t-1), not occurs(A, t) : post(A, X, V).

```

Listing 2: Core for sequential and parallel encodings for planning

All planning algorithms share a common incremental encoding using the incmode for clingo as seen in Listing 2 (Gebser et al. 2014). The program is divided into three subprograms: *base*, *step(t)* and *check(t)*. The *base* subprogram contains the static part of the encoding, which is only grounded once. Line 2 defines the initial state s_0 , where the predicate $holds(X, V, t)$ is used to represent fluents X that have value V at timestep t .

check(t) is a parametrized subprogram starting at $t = 0$. The integrity constraint in Line 4 checks if all fluents X defined in $goal(X, V)$ has value V at timestep t , i.e. $s_*(X) = s_t(X) \forall X \in dom(s_*)$. *Query(t)* is true only for the latest integer t .

step(t) is grounded after *check* fails for each timestep t starting at $t = 1$. Lines 6 defines holds for times step t , giving fluent X one value V from its domain. and *occurs(A, t)* in Line 7 are all possible actions in timestep t obeying rules 8 and 9. Line 8 ensures that the postcondition of an action is applied. Note that the postcondition is applied in the same timestep as the action occurs. It also assert the confluence of any set of action applied in parallel since *holds* and *post* has to be true. Line 9 makes sure that fluents not affected by any action keep their value from the previous timestep. So far, the common core handles the transitions from one state to the next in regards of the postcondition of applied actions. So far, the common part takes care of matching sucesor states to postconditions of applied actions.

2.2.1 encoding for sequential plans

```
1 :- occurs(A, t), prec(A, X, V), not holds(X, V, t-1).
2 :- #count{A : occurs(A, t)} > 1.
```

Listing 3: Extension for sequential plans

Starting with the extension for sequential plans as seen in Listing 3, Line 2 ensures that the precondition of an applied action is met in $t - 1$ and Line 3 only allows one action to occur per timestep.

Example 5. Running this with our example, we get three possible sequential plans.

```
Answer: 1
occurs(getKey,1) occurs(lightOut,2) occurs(openDoor,3) occurs(goOut,4) occurs(closeDoor,5)
Answer: 2
occurs(getKey,1) occurs(openDoor,2) occurs(lightOut,3) occurs(goOut,4) occurs(closeDoor,5)
Answer: 3
occurs(lightOut,1) occurs(getKey,2) occurs(openDoor,3) occurs(goOut,4) occurs(closeDoor,5)
```

Listing 4: Output from clingo for sequential encoding

2.2.2 encoding for $\forall$ -step plans

```
1 :- occurs(A, t), prec(A, X, V), not holds(X, V, t-1).
2 :- occurs(A, t), prec(A, X, V), not post(A, X, _), not holds(X, V, t).
3 single(X, t) :- occurs(A, t), prec(A, X, V1), post(A, X, V2), V1 != V2.
4 :- single(X, t), #count{A : occurs(A, t), post(A, X, V)} > 1.
```

Listing 5: Extension for $\forall$ -step plans

Bibliography

- Helmert, M. (2006). “The Fast Downward Planning System”. In: *Journal of Artificial Intelligence Research* 26, pp. 191–246.
- Gebser, Martin et al. (May 2014). *Clingo = ASP + control: Preliminary report*. Tech. rep. 1405.3694. arXiv. URL: <https://arxiv.org/abs/1405.3694v1>.